AI-driven Dynamic Weighted Edge Computing Scheduler

GROUP B6

Rishitha C - CB.AI.U4AID24109

Divya Rithanya S - CB.AI.U4AID24113

Karthikeya - CB.AI.U4AID24163

Kaushal S - CB.AI.U4AID24168

10 October 2025

Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Problem Statement

- MEC servers have limited computing power and energy resources.
- Tasks arrive randomly and unpredictably, making scheduling complex.
- Efficient task scheduling is critical to ensure timely task completion.
- Poor scheduling leads to lower system performance and reduced user satisfaction.
- **Main Challenge:**
  - Designing a scheduler that balances task completion, deadlines, and energy efficiency

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Objectives

- Develop an intelligent scheduler for heterogeneous servers.
- Implement Lightweight Q-learning to dynamically adjust scheduling weights.
- Optimize for:
  - Task completion before deadlines.
  - Reduce total energy consumption.
- Compare performance against traditional scheduling methods.

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Literature Review

S.No.	Name	Publisher|Year	Inference	Cons.
1	Research on Task Scheduling Methods in 5G Mobile Edge Computing Environments	IEEE-2024	Improved task completion vs FCFS, EDF, SJF by using weighted factors	Fixed weights; not adaptive
2	Resource Calendaring for Mobile Edge Computing in 5G Networks	IEEE-2021	Efficient resource utilization, Reduced latency & Better load balancing	Extra overhead to maintain calendar

Literature Review

S.No.	Name	Publisher|Year	Inference	Cons.
3	Resource Adaptive Automated Task Scheduling Using Deep Deterministic Policy Gradient in Fog Computing	IEEE-2024	Reduced latency & energy with adaptive RL	High training complexity
4	Faashouse: Sustainable Serverless Edge Computing Through Energy-Aware Resource Scheduling	IEEE-2024	Improved utilization & sustainability in serverless MEC	Restricted to serverless; not general MEC

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Research Gap & Novelty

Research Gap

- FCFS, EDF, SJF → simple but inefficient in MEC.
- DWECS improves scheduling but uses static weights.
- Advanced RL methods exist but are complex and resource-heavy.

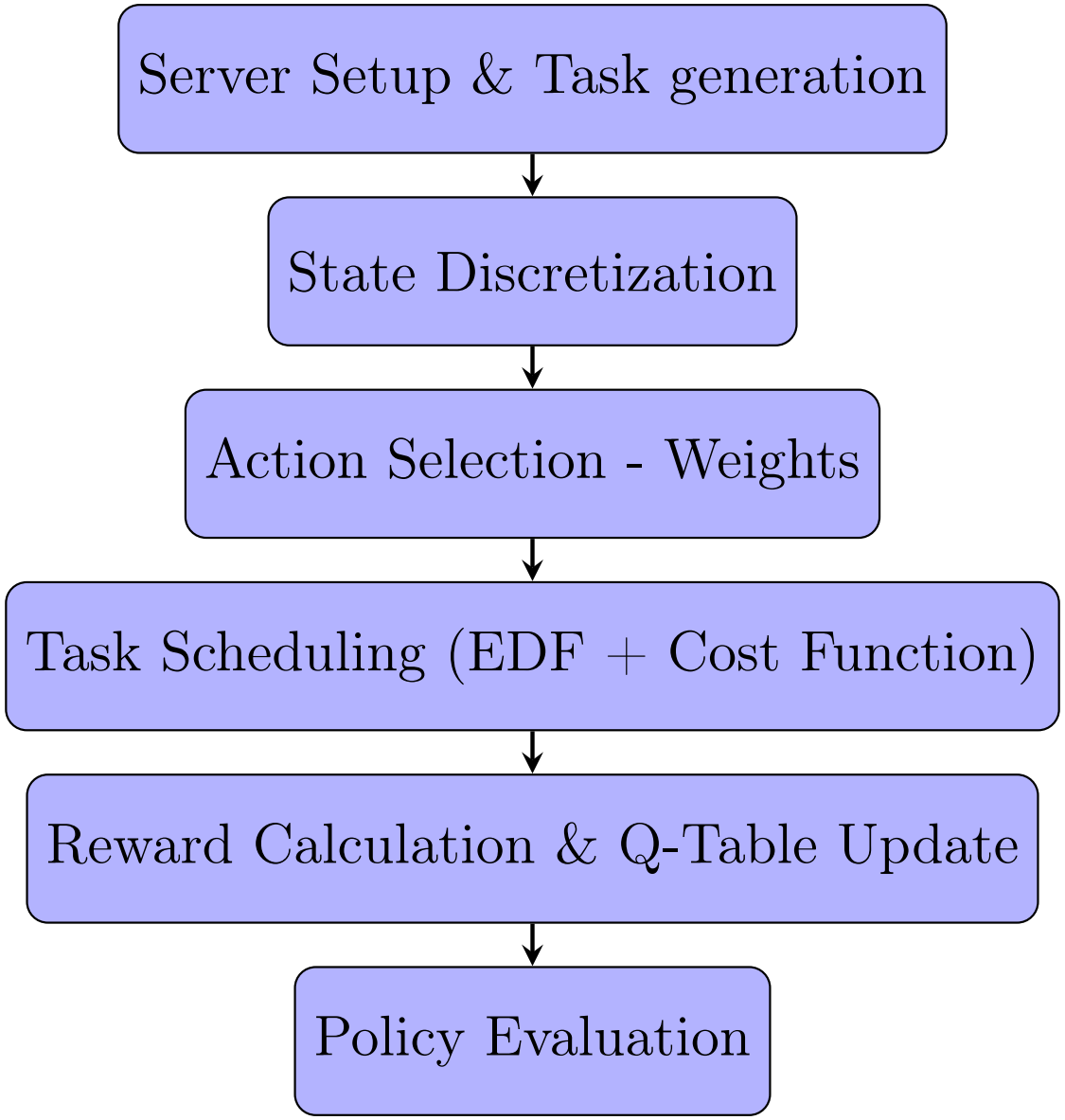
Novelty

- Dynamic weight adjustment using lightweight Q-learning.
- Balances task completion & energy adaptively.
- Lightweight design → suitable for real-time MEC.
- Provides fair comparison with FCFS, EDF, SJF, and DWECS.
- Adaptive to dynamic task arrivals and heterogeneous MEC servers.

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Methodology



Methodology

Server Setup & Task Generation

- Create 3 heterogeneous servers. And generate random tasks with varying computation loads and deadlines to simulate a real-time MEC environment.

State Discretization

- Represent the system state using average task urgency and average task load.
- Reduces the state space for lightweight Q-learning.

Action Selection - Weights

- Choose scheduling weights for time vs energy trade-off.
- Use epsilon-greedy policy: explore randomly sometimes, otherwise exploit the best-known action.

Methodology

Task Scheduling (EDF + Cost Function)

- Apply Earliest Deadline First (EDF) to order tasks.
- Assign tasks to servers minimizing a weighted cost function (time + energy).

Reward Calculation & Q-Table Update

- Reward = tasks completed − penalty × energy consumed.
- Update Q-table using lightweight Q-learning for adaptive scheduling.

Policy Evaluation

- Measure task completion rate, energy consumption, and execution efficiency.
- Compare the results.

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Implementation

Server details

- Server 1: GHz = 2.0, Power = 10 W
- Server 2: GHz = 3.0, Power = 15 W
- Server 3: GHz = 1.5, Power = 7.5 W

Task Generation

- Batch size: 10 tasks per batch
- Load: 2–15 billion cycles
- Deadline: 2000–20000 ms
- Arrival: Sequential

Implementation

State Space

Compute avg_deadline → Urgency_Bin: 1 if <6000 ms, else 0

Compute avg_load → Load_Bin: int(avg_load // 3e9)

Load Bin: 0 (light), 1 (medium), 2 (heavy) based on average task load.

Action Space

Predefined weight combinations for cost function

Each action decides how scheduler balances completion vs energy.

Implementation

EDF Ordering: Tasks sorted by prioritizing the earliest deadline.

Execution Time

$$T_{\text{exec}} = \frac{\text{Load (cycles)}}{\text{GHz} \times 10^9} \times 1000 \tag{1}$$

Energy Formula

$$E = \frac{T_{\text{exec}}}{1000} \times P \tag{2}$$

Server Availability

$$FinishTime = AvailableTime + ExecutionTime \tag{3}$$

Implementation

Normalize time and energy

Cost function

$$\text{Cost} = w_{\text{time}} \cdot \text{NormTime} + w_{\text{energy}} \cdot \text{NormEnergy} \tag{4}$$

- This ensures the task is assigned to the server that best balances execution time and energy according to the current weights.
- Logging - a record of each task is saved for reporting or debugging.

Outputs

- Completion Status
- Energy Consumed
- Server assignments

Implementation

Training: 10000 episodes run.

- Lightweight Q-learning with small Q-table
- Epsilon-greedy policy for exploration vs exploitation

Reward Function

$$\text{Reward} = \text{Tasks Completed} - 0.002 \times \text{Energy Consumed (J)} \tag{5}$$

Q-Table Update

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{6}$$

Evaluation: Choose the task based on exploitation.

- Learned policy compared against EDF, FCFS, SJF.
- Measures: Task completion rate & Energy consumption

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Results & Discussion

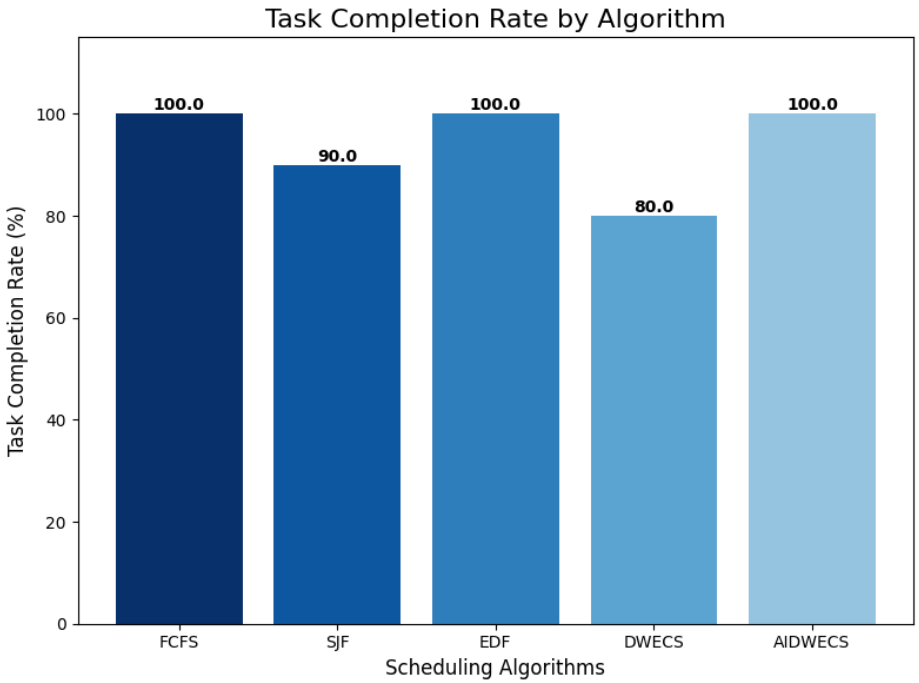
Tabela 1: 1st Task Batch for Evaluation

ID	Load (Cycles)	Deadline (ms)	Arrival Time (ms)
1	3×10^9	2000	0
2	10×10^9	10000	500
3	2×10^9	3000	1000
4	4×10^9	4500	1500
5	12×10^9	15000	2000
6	3.5×10^9	7000	3000
7	5×10^9	7500	4000
8	2.5×10^9	8000	5000
9	8×10^9	14000	5500
10	6×10^9	13000	6000

Results & Discussions

Tabela 2: Comparison of Task Completion Rate

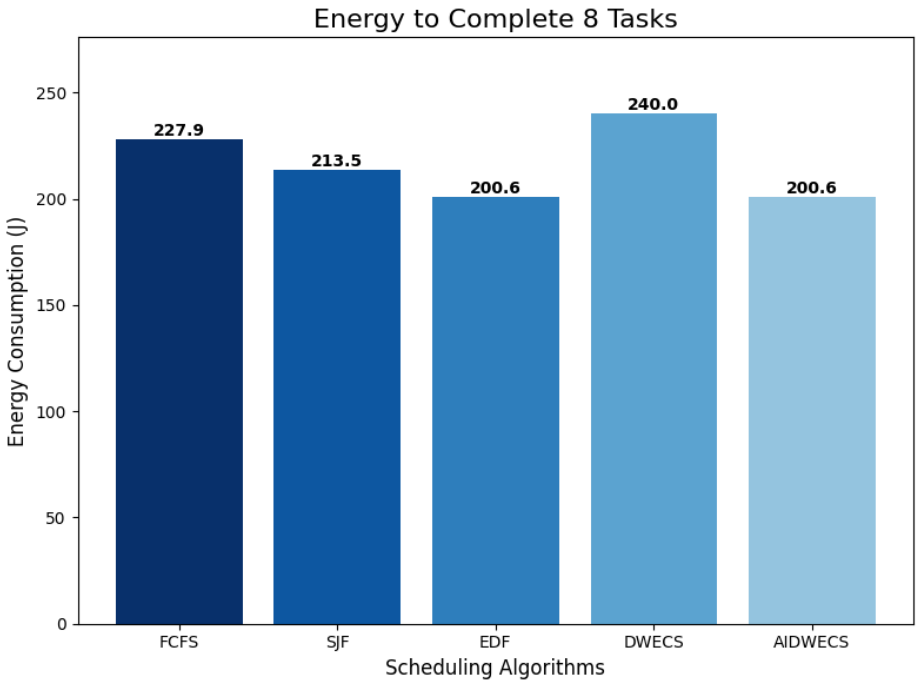
Scheduler	Task Completion (%)
FCFS	100.0
SJF	90.0
EDF	100.0
DWECS	80.0
AIDWECS	100.0



Results & Discussion

Tabela 3: Comparison of Energy Consumption for 8 Tasks

Scheduler	Energy Consumption (J)
FCFS	227.9
SJF	213.5
EDF	200.6
DWECS	240
AIDWECS	240.6



Results & Discussion

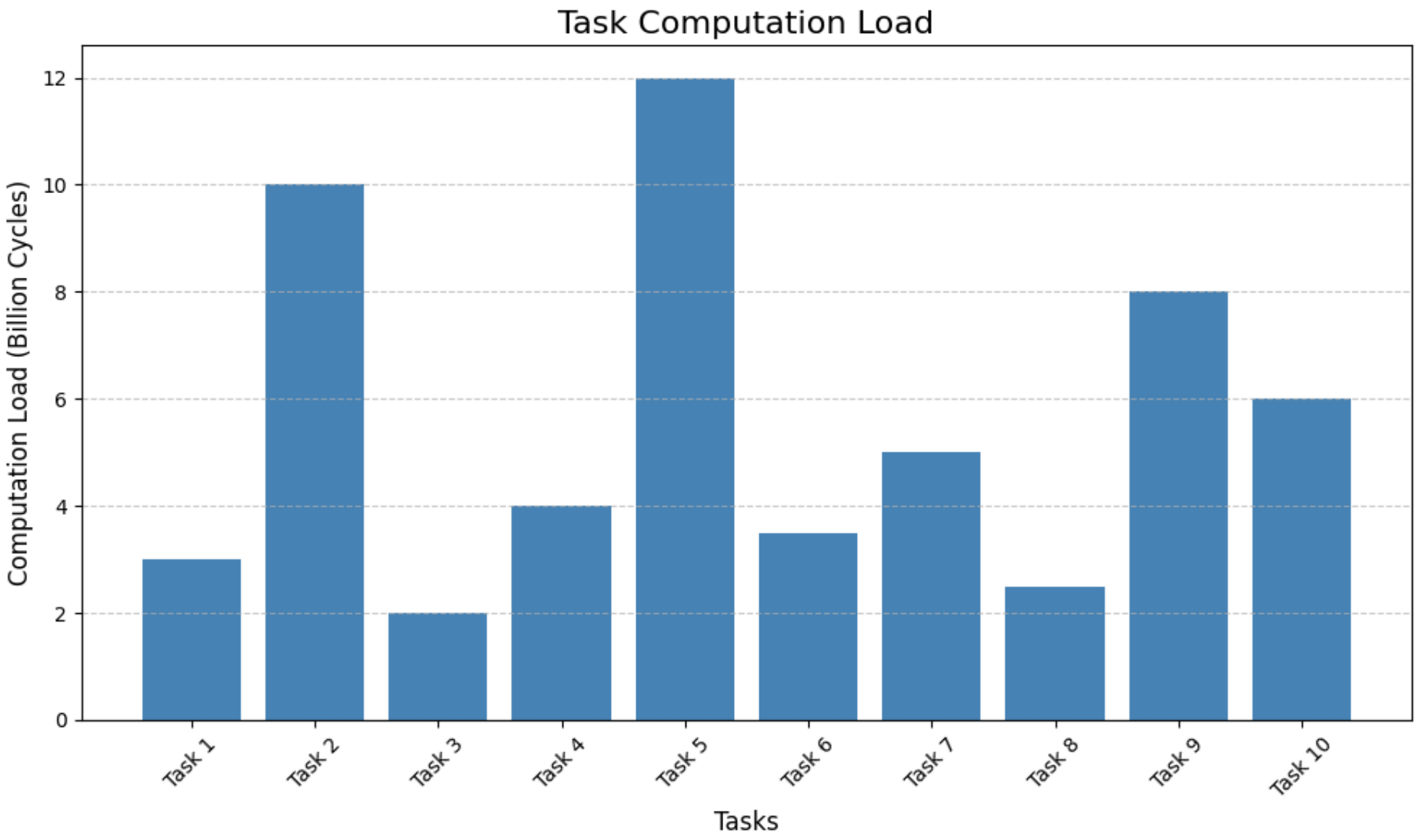


Figura 3: Task Computation Load

Results & Discussion

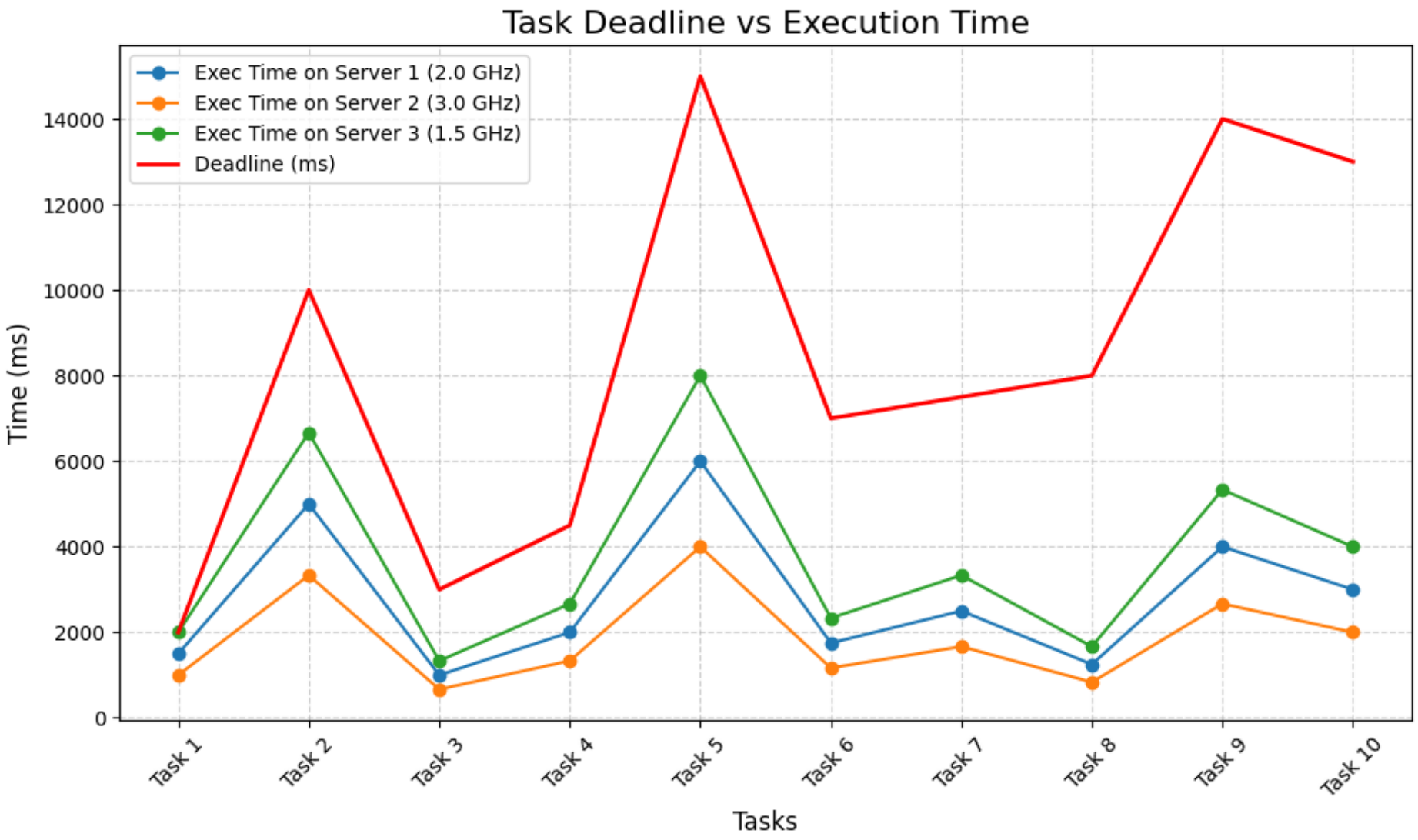


Figura 4: Task Deadline Vs. Execution Time

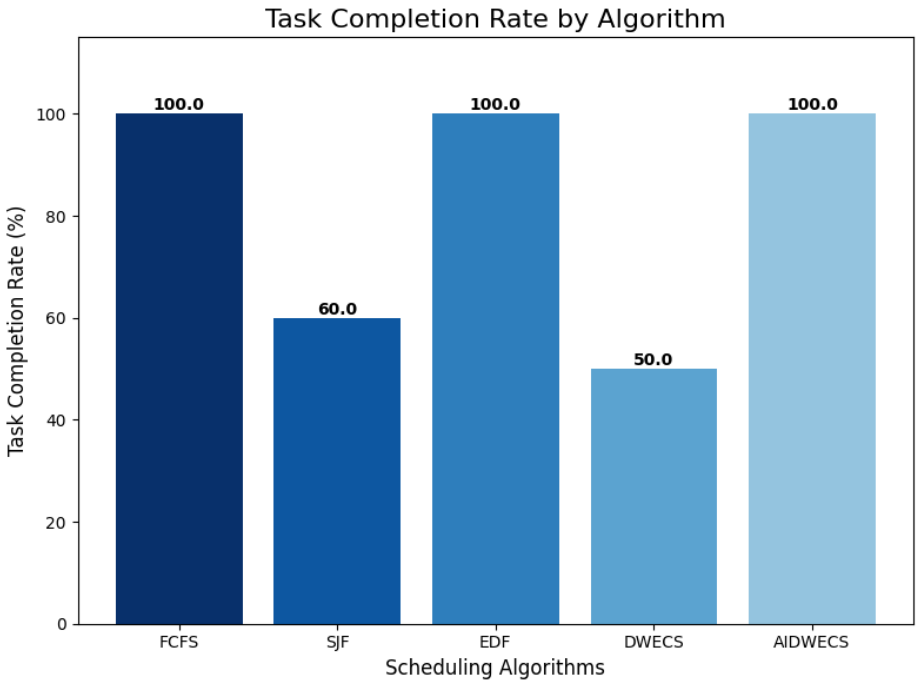
Tabela 4: 2nd Task Batch for Evaluation

ID	Load (Cycles)	Deadline (ms)	Arrival Time (ms)
1	4×10^9	2500	0
2	8×10^9	9000	0
3	6×10^9	5000	0
4	3×10^9	2800	200
5	5×10^9	6000	500
6	10×10^9	12000	2000
7	2×10^9	8000	3500
8	7×10^9	11000	4500
9	3.5×10^9	9500	6000
10	9×10^9	16000	7000

Results & Discussions

Tabela 5: Comparison of Task Completion Rate

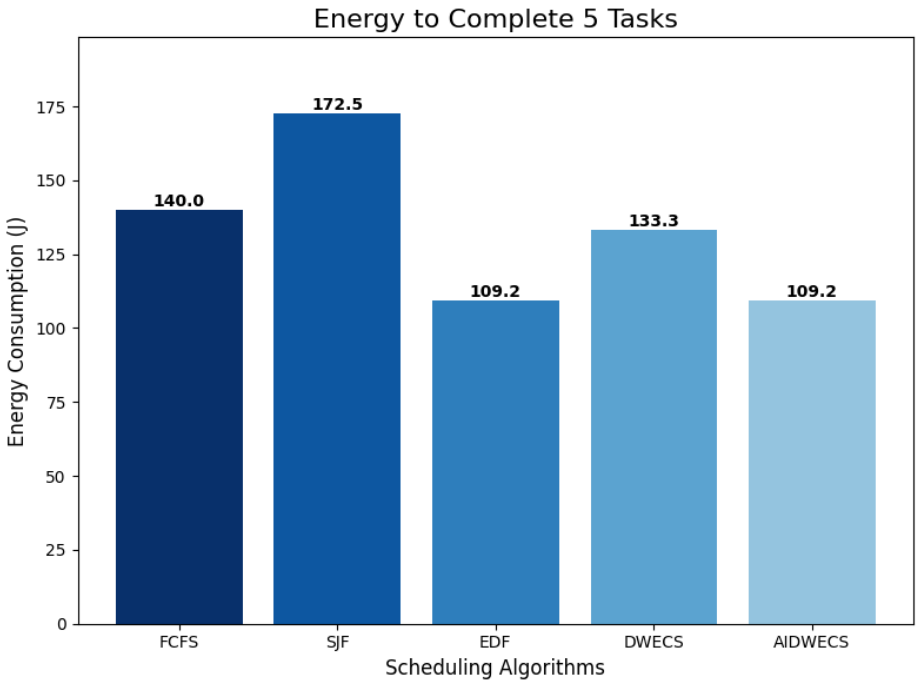
Scheduler	Task Completion (%)
FCFS	100.0
SJF	60.0
EDF	100.0
DWECS	50.0
AIDWECS	100.0



Results & Discussion

Tabela 6: Comparison of Energy Consumption for 5 Tasks

Scheduler	Energy Consumption (J)
FCFS	140.0
SJF	172.5
EDF	109.2
DWECS	133.3
AIDWECS	109.2



Results & Discussion

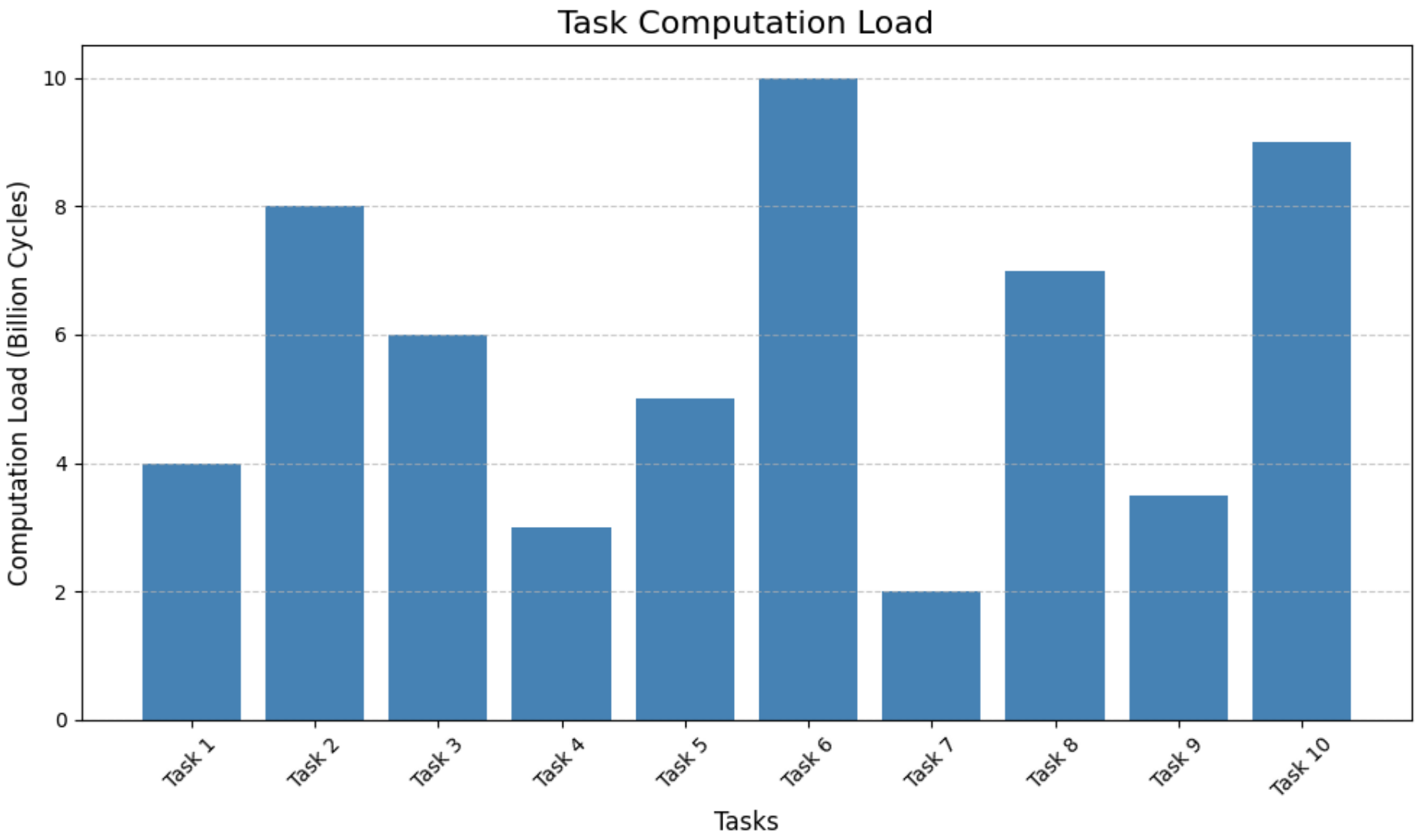


Figura 7: Task Computation Load

Results & Discussion

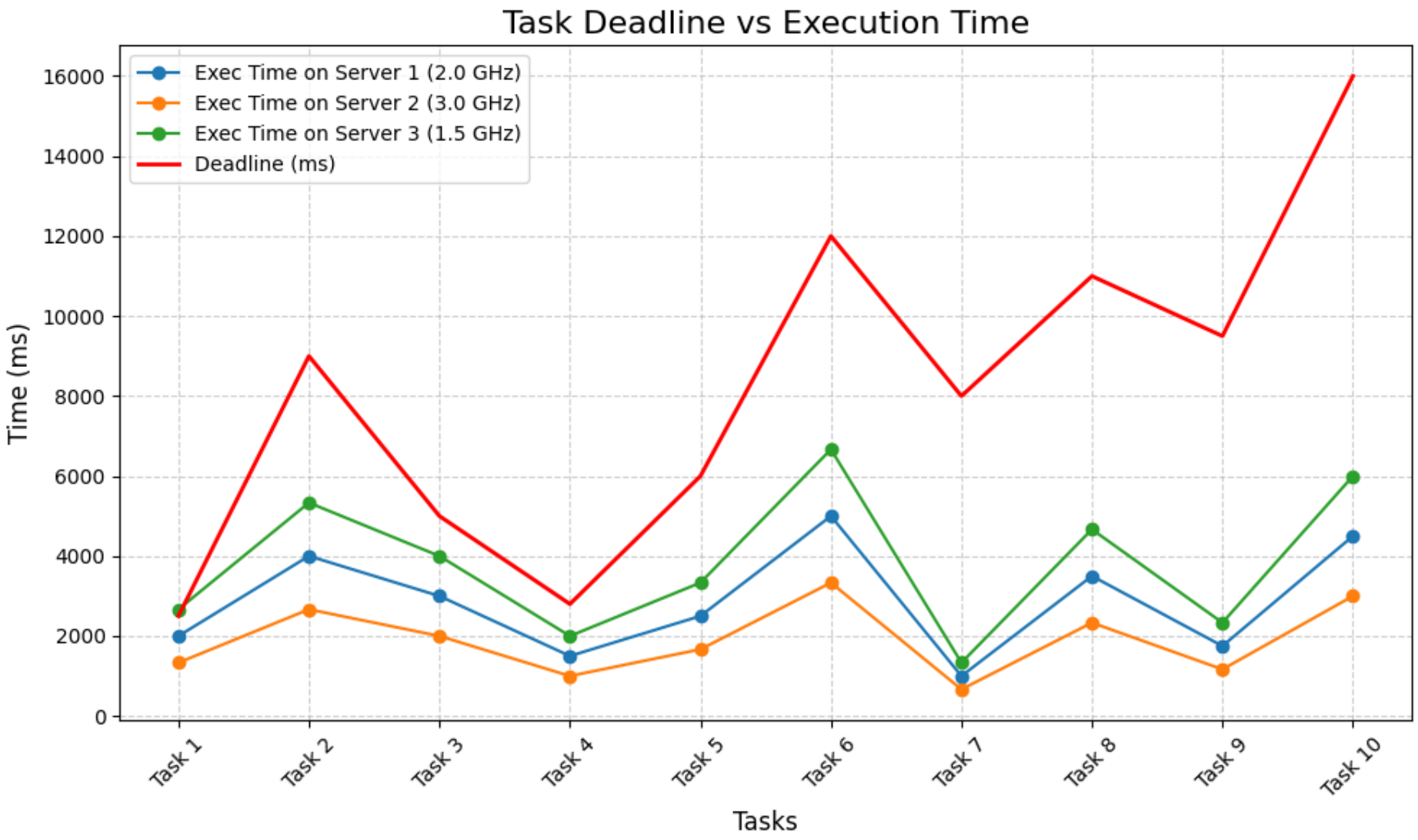


Figura 8: Task Deadline Vs. Execution Time

Results & Discussion

Task Scheduling Performance Comparison

- Schedulers Compared: EDF, FCFS, SJF, DWECS, AIDWECS.
- Metrics Evaluated: Task Completion Rate(%), Total Energy Consumption (J) & Task Deadline vs. Execution Time.

Key Observations:

- EDF and AIDWECS achieve the highest completion rates and lowest energy consumption for the tested task batch.
- FCFS, SJF & DWECS performs comparitively worse.
- The scheduling strategies EDF, AIDWECS improves the task success.
- But since both EDF and AIDWECS show same results, we will compare them again.

Results & Discussion

Tabela 7: Comparing EDF & AIDWECS Stress Test

Scheduler	Server	Deadline	Exec Time	Avl At	Finish Time	Violation
EDF	Server 2	12000	1333.33	13000	14333.33	2333.33
AIDWECS	Server 1	12000	2000	11000	13000	1000

- In a high-congestion stress test, AIDWECS mitigated the deadline violation to 1,000 ms, while the static EDF failed with a 2,333 ms delay.
- Conclusion: This proves AIDWECS provides 57% better system reliability by adapting to resource congestion, a critical capability EDF lacks.

Tabela 8: Worst-Case Time Complexity Comparison (N: Tasks, M: Servers)

Algorithm	**Time Complexity**	**Validation/Justification**
FCFS, SJF, EDF	$O(N \log N + N \cdot M)$	Complexity is dominated by the initial sort of N tasks ($O(N \log N)$), plus the cost of checking M servers for each of the N tasks ($O(N \cdot M)$).
DWECS	$O(N \log N + N \cdot M)$	Requires $O(N \cdot M)$ to find max values for cost normalization. Uses $O(N \log N)$ for the initial sort. Main scheduling loop is $O(N \cdot M)$.
AIDWECS	$O(N \log N + N \cdot M)$	Pre-scheduling includes an $O(1)$ Q-table lookup. Complexity is dominated by sorting ($O(N \log N)$) and the necessity of $O(N \cdot M)$ pre-computation and server selection.

Tabela 9: Worst-Case Space Complexity Comparison (N: Tasks, M: Servers)

Component	**Space Complexity**	**Validation/Justification**
All Schedulers	$O(N)$	The primary memory usage scales with the number of tasks being stored (e.g., deep copy of tasks and output logs). Server state is a constant $O(M)$.
AIDWECS (Q-Table)	$O(1)$	The state space is fixed by the small, constant number of discretization bins (e.g., Urgency $\times$ Load). Since the number of states and actions is fixed, the storage is constant.

Tabela 10: Per-Task Effort (Constant Factor) Analysis in $O(N \cdot M)$ Loop

Algorithm	**Complexity per Task**	**Constant Factor Analysis (Decision)**
FCFS, SJF, EDF	$O(M)$	**Lowest Constant Factor**. Decision is a simple comparison of finish times across M servers, requiring minimal arithmetic per server.
DWECS	$O(M)$	**Higher Constant Factor**. Involves calculating multiple metrics (time, energy, tardiness), normalizing them (division), and then summing the weighted values for each of the M servers.
AIDWECS	$O(M)$	**Highest Constant Factor**. Performs the same complex multi-metric calculation and normalization as DWECS, plus an $O(1)$ Q-table lookup for weights before the loop, leading to the highest operational cost per task-server pair.

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

Conclusion

- Best Performing Schedulers: EDF and AIDWECS consistently achieve the highest task completion rates and lower energy consumptions.
- Energy Efficiency: AIDWECS & EDF shows the least energy consumption to complete a fixed number of tasks.
- Effectiveness of Intelligent Scheduling:
  - Though AIDWECS & EDF gives the same results, AIDWECS performs well by demonstrating superior adaptive intelligence in high-congestion states, mitigating critical deadline violations by 57% where EDF's rigid policy failed.
- Using deadline-aware and cost-aware schedulers improves system performance with the least possible energy consumption, making them ideal for heterogeneous edge environments.

Table of Contents

1. Problem Statement
2. Objectives
3. Literature Review
4. Research Gap & Novelty
5. Methodology
6. Implementation
7. Results & Discussion
8. Conclusion
9. References

References I

[1] G. Zhu, T. Liu, Y. Guo, D. Xia, D. Lv and X. Zhang, "Research on Task Scheduling Methods in 5G Mobile Edge Computing Environments,"2024 6th International Conference on Energy, Power and Grid (ICEPG), Guangzhou, China, 2024, pp. 1789-1794, doi: 10.1109/ICEPG63230.2024.10775421.

[2] B. Xiang, J. Elias, F. Martignon and E. Di Nitto, "Resource Calendaring for Mobile Edge Computing in 5G Networks,"ICC 2021 - IEEE International Conference on Communications, Montreal, QC, Canada, 2021, pp. 1-6, doi: 10.1109/ICC42927.2021.9500998.

[3] P. Choppara and S. S. Mangalampalli, "Resource Adaptive Automated Task Scheduling Using Deep Deterministic Policy Gradient in Fog Computing,"in IEEE Access, vol. 13, pp. 25969-25994, 2025, doi: 10.1109/ACCESS.2025.3539606.

[4] M. S. Aslanpour, A. N. Toosi, M. A. Cheema and M. B. Chhetri, "Faashouse: Sustainable Serverless Edge Computing Through Energy-Aware Resource Scheduling,"in IEEE Transactions on Services Computing, vol. 17, no. 4, pp. 1533-1547, July-Aug. 2024, doi: 10.1109/TSC.2024.3354296.

[5] J. Chen, Y. Mao, Y. Zhang, "Energy-Efficient Task Scheduling for Multi-Access Edge Computing,"IEEE Transactions on Green Communications and Networking, vol. 6, no. 3, pp. 1054-1065, Sept. 2022, doi: 10.1109/TGCN.2022.3156789.

[6] S. K. Sharma, S. Chatterjee and S. Misra, "Optimal Task Scheduling in Edge-Cloud Environments for Latency Minimization,"IEEE Systems Journal, vol. 16, no. 2, pp. 2000-2011, June 2022, doi: 10.1109/JSYST.2021.3098765.

[7] F. Bonomi, R. Milito, J. Zhu, S. Addepalli, "Fog Computing and Its Role in the Internet of Things,"in Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, Helsinki, Finland, 2012, pp. 13-16.

References II

[8] W. Zhang, Y. Wen, J. Wu, H. Li, "Task Scheduling in Mobile Edge Computing for Internet of Things,"IEEE Internet of Things Journal, vol. 6, no. 3, pp. 5540-5550, June 2019, doi: 10.1109/JIOT.2019.2904792.

[9] L. Wang, K. Yang, M. Li, "AI-Assisted Resource Allocation and Task Scheduling in Edge Computing,"Future Generation Computer Systems, vol. 140, pp. 86-98, Jan. 2023, doi: 10.1016/j.future.2022.09.012.

[10] X. Xu, J. Zhang, F. Xia, "Heuristic and Metaheuristic Approaches for Task Scheduling in Cloud and Edge Computing,"Journal of Network and Computer Applications, vol. 180, 103033, 2021, doi: 10.1016/j.jnca.2020.103033.

[11] M. Satyanarayanan, P. Bahl, R. Caceres, N. Davies, "The Case for VM-Based Cloudlets in Mobile Computing,"IEEE Pervasive Computing, vol. 8, no. 4, pp. 14-23, Oct.-Dec. 2009, doi: 10.1109/MPRV.2009.82.

[12] C. Xu, X. Liu, Q. Chen, "Deep Reinforcement Learning-Based Task Scheduling in Multi-Access Edge Computing,"IEEE Access, vol. 9, pp. 112345-112358, 2021, doi: 10.1109/ACCESS.2021.3105678.

[13] Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective,"IEEE Communications Surveys Tutorials, vol. 19, no. 4, pp. 2322-2358, Fourthquarter 2017, doi: 10.1109/COMST.2017.2745201.

THANK YOU!!

Q & A